

Lesson 1.5: Teamwork Standards

Vấn Đề Thường Gặp Trong Teamwork & Chuẩn Hóa Code Quality

- **Khóa học:** NestJS Practical Masterclass: Zero to Hero
- **Dự án thực chiến:** Real-Time Social Chat Application
- **Thời lượng:** ~3 - 5 phút

3 Thách Thức Khi Làm Việc Nhóm

- **Code Style Không Thống Nhất:**
Mỗi dev một kiểu format (tab/space, dấu phẩy).
- **Cảnh Báo Lint Bị Ngó Lơ:**
Để lọt kiểu `any`, thừa biến, rò rỉ `console.log`.
- **Commit Message Tùy Tiện:**
Đặt tên commit vô nghĩa như `"fix bug"`, `"update"`.

TEAMWORK CODE QUALITY COMPARISON

UNSTANDARDIZED CODE

```
var x = 1 // no semicolon
const user:any={name:"A"}
console.log(user) // leftover
function test(){return}
```

- Git Diff 500+ lines (Format noise)
- Code Review tốn nỗ lực vô ích
- Commit message: "fix bug", "up"
- Nguy cơ cao rò rỉ bug lên Prod

NESTJS STANDARDIZED CODE

```
const user: UserEntity = {
  name: 'Alex',
};
function test(): void {}
```

- Git Diff tập trung đúng 100% logic
- Auto-format khi lưu (Save)
- Commit chuẩn Conventional
- Tự động chặn code rác với Husky

Hậu Quả Của Việc Thiếu Chuẩn Hóa

Lãng Phí Nỗ Lực Code Review

- Tech Lead phải bắt từng lỗi thụt lề, dấu chấm phẩy.
- Giảm 50% hiệu suất review logic nghiệp vụ thực tế.

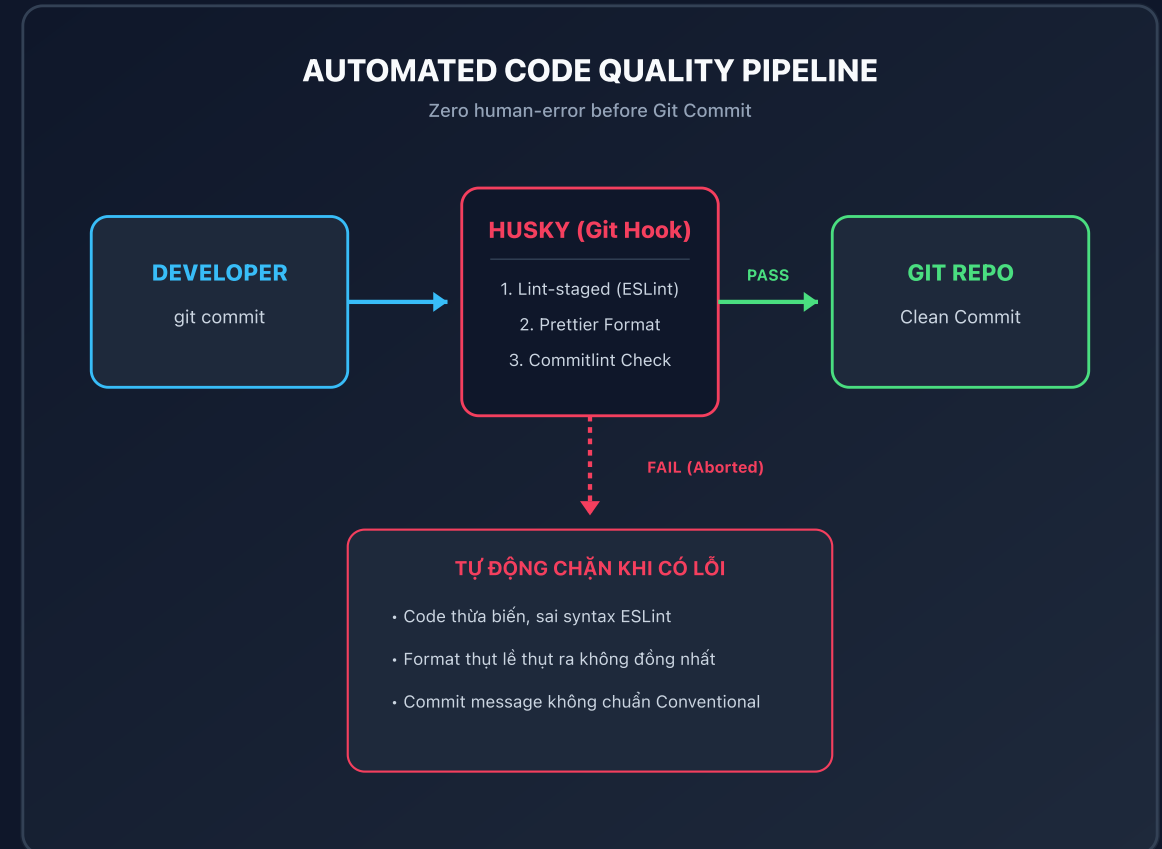
Git Diff Bị Nhiều Sạn

- Sửa 1 dòng code nhưng Git ghi nhận thay đổi 500 dòng do auto-format lệch nhau.
- Khó truy vết lịch sử commit khi cần Rollback.

Thực tế ngành: 80% thời gian của lập trình viên là đọc & bảo trì mã nguồn của đồng nghiệp.

Giải Pháp Tự Động Hóa (Automation First)

- **Ngăn chặn từ gốc (Pre-commit Hook):**
Chặn đứng code không chuẩn ngay trên máy dev trước khi gõ lệnh commit.
- **3 Lớp Kiểm Soát:**
 - i. **ESLint:** Quét lỗi cú pháp & code smells.
 - ii. **Prettier:** Tự động định dạng code thống nhất.
 - iii. **Commitlint:** Ép tuân thủ Conventional Commits.



Bộ Công Cụ Triển Khai Trong Module 1

Code Formatter & Linter

- **ESLint:** Phân tích tĩnh mã nguồn TypeScript.
- **Prettier:** Tự động định dạng code nhất quán khi lưu.

Git Automation & Commit Rules

- **Husky:** Tự động kích hoạt script khi chạy lệnh Git.
- **Lint-staged:** Chỉ quét lint trên các tệp tin đang được commit.
- **Commitlint:** Kiểm tra định dạng commit message.

Tổng Kết & Bước Tiếp Theo

Key Takeaways:

1. **Chuẩn hóa Code Quality** là bắt buộc cho dự án Enterprise & Teamwork.
2. Tự động hóa quá trình kiểm tra format & linting thông qua **Git Hooks**.
3. Đảm bảo mã nguồn luôn sạch trước khi đẩy (push) lên Git repository.

Bài học tiếp theo: Lesson 1.6 - Git Hooks & Husky

Chúng ta sẽ bắt tay vào **cấu hình Husky để tự động kiểm tra code** mỗi khi gõ lệnh `git commit` !